École Marocaine des Sciences de l'Ingénieur

Filière 4DSI – Groupe G11

RAPPORT DE PROJET PFA

Application web de gestion d'école

Réalisé par : ANAS CHBIRA

Année universitaire : [2026–2027]

Résumé

Ce rapport présente la conception et la réalisation d'une application web de gestion d'école, développée pour mon PFA en 4DSI. L'application s'adresse à trois profils : administrateur, professeur et élève.

Elle permet de gérer les comptes, les classes, les matières, les emplois du temps, les notes, les absences, les mesures disciplinaires, ainsi que les cours et exercices publiés par les enseignants. Un calendrier hebdomadaire navigable affiche les cours par date, avec gestion des annulations et des séances exceptionnelles.

Le backend repose sur Spring Boot, Spring Security avec authentification par jeton JWT, et une base MySQL accessible via JPA. Le frontend est une application React construite avec Vite. Le rapport détaille l'analyse des besoins, la conception, la réalisation, la sécurité, les tests, les difficultés rencontrées et les perspectives d'évolution.

Mots-clés : gestion scolaire, Spring Boot, React, JWT, JPA, API REST.

Table des matières

Résumé **i**

Introduction **1**

1 Présentation du projet **2**

1.1 Sujet et problématique 2

1.2 Objectifs 2

1.3 Périmètre 2

1.4 Méthode de travail 3

2 Analyse des besoins **4**

2.1 Acteurs 4

2.2 Besoins fonctionnels 4

2.3 Règles de gestion 5

2.4 Besoins non fonctionnels 5

3 Choix technologiques **7**

4 Conception **9**

4.1 Architecture générale 9

4.2 Modèle de données 9

4.3 Héritage JPA 11

4.4 Conception du calendrier 12

4.5 Évolution du modèle 12

5 Réalisation du backend **13**

5.1 Structure 13

5.2 Authentification et autorisation 13

5.3 Requêtes avec filtres optionnels 13

5.4 Calendrier dynamique 14

5.5 Actions en masse . . . 14
5.6 Protection des données dans les réponses JSON . . . 14

6 Réalisation du frontend 15
6.1 Organisation du code . . . 15
6.2 Authentification côté client . . . 15
6.3 Composants réutilisables . . . 15
6.4 Interface et thème . . . 16
6.5 Découpage du tableau de bord professeur . . . 16

7 Fonctionnalités par rôle 17
7.1 Administrateur . . . 17
7.2 Professeur . . . 19
7.3 Élève . . . 19

8 Sécurité et protection des données 21
8.1 Mécanismes en place . . . 21
8.2 Contrôles de propriété . . . 21
8.3 Points d'attention connus . . . 22

9 Tests et validation 23
9.1 Parcours de bout en bout . . . 23
9.2 Tests de sécurité . . . 24
9.3 Cas limites . . . 24
9.4 Limite des tests . . . 24

10 Difficultés rencontrées et solutions 25

11 Limites et perspectives d'évolution 27
11.1 Limites actuelles . . . 27
11.2 Perspectives fonctionnelles . . . 27
11.3 Perspectives techniques . . . 27
11.4 Accessibilité et inclusion . . . 28

12 Conclusion 29

13 Webographie 30

14 Annexes 31

14.1 Annexe A : structure du projet 31
14.2 Annexe B : principaux endpoints de l'API 31

Introduction

Ce rapport décrit la conception et la réalisation d'une application web de gestion d'école, développée durant mon stage en filière 4DSI, groupe G11 [période et durée du stage à compléter].

Le sujet m'a été proposé par l'encadrement : permettre à une école de gérer ses élèves, ses enseignants, ses emplois du temps et le suivi pédagogique. Le projet couvre toute la chaîne technique, de la base de données à l'interface utilisateur.

Le rapport suit la démarche du projet. Il présente d'abord le sujet et l'analyse des besoins, puis les choix technologiques et la conception. Il détaille ensuite la réalisation du backend et du frontend, les fonctionnalités par rôle, la sécurité et les tests. Il se termine par les difficultés rencontrées, les limites de la solution et ses perspectives.

Chapitre 1

Présentation du projet

1.1 Sujet et problématique

Le sujet est le développement d'une application web de gestion d'école. Dans un établissement, les informations pédagogiques et administratives (emplois du temps, notes, absences, supports de cours) sont souvent dispersées entre plusieurs outils ou documents papier.

La problématique est la suivante : comment centraliser ces informations dans une seule application, tout en garantissant que chaque utilisateur n'accède qu'aux données qui le concernent ?

1.2 Objectifs

— Offrir à chaque profil (administrateur, professeur, élève) un espace adapté à ses besoins.

— Gérer le cycle pédagogique : classes, matières, enseignements, emplois du temps, notes, absences et mesures disciplinaires.

— Permettre aux professeurs de publier des cours et des exercices, et aux élèves d'y répondre.

— Fournir à l'administration des outils de gestion en masse : filtrage, déplacement, export.

— Protéger l'accès par authentification et contrôle des rôles.

1.3 Périmètre

Le cahier des charges initial incluait un système de clubs d'élèves (création par pétition de dix membres, événements réservés aux membres) et un circuit de validation des mesures disciplinaires proposées par les professeurs. Ces deux volets ont été volontairement retirés pour tenir le délai, afin de livrer un noyau fonctionnel complet et testé.

Les pièces jointes sur les cours et les réponses, ainsi que le téléversement de photos de profil, sont reportés en perspective (section 11).

1.4 Méthode de travail

Le développement a été itératif, par étapes courtes : modèle de données, accès aux données, sécurité, contrôleurs, puis interface. Chaque étape a été vérifiée avant la suivante, d'abord avec Postman, ensuite directement depuis l'interface.

Outils utilisés : Spring Tools Suite 5 (Eclipse), Maven, MySQL, Postman, Node.js et npm, ainsi que les outils de développement du navigateur pour le débogage.

Chapitre 2

Analyse des besoins

2.1 Acteurs

Acteur	**Rôle dans l'école**	**Droits principaux**
Administrateur	Gère l'ensemble de l'établissement	Création et modification des comptes, classes, matières, enseignements, créneaux et mesures disciplinaires ; accès à toutes les données.
Professeur	Enseigne une ou plusieurs matières dans plusieurs classes	Publication de cours et d'exercices, notation, saisie des absences et consultation de son calendrier.
Élève	Appartient à une classe	Consultation en lecture seule de ses données et réponse aux exercices.

Table 2.1 – Acteurs et droits principaux

2.2 Besoins fonctionnels

Élève

— Consulter le calendrier hebdomadaire de sa classe, avec le nom du professeur et la salle.

— Consulter ses notes, ses absences (avec horaires et statut justifié ou non) et ses mesures disciplinaires.

— Consulter les cours et exercices de sa classe et répondre aux exercices.

— Voir son profil, sans pouvoir le modifier.

Professeur

— Consulter son calendrier hebdomadaire et y ajouter des remarques datées.

— Voir la liste des élèves de chaque classe qu'il enseigne, avec leurs absences dans sa matière.

— Publier des cours et des exercices, et lire les réponses des élèves.

— Noter les élèves et marquer une absence avec date et plage horaire.

Administrateur

— Créer et modifier les comptes, classes, matières et enseignements (professeur, classe, matière).

— Définir l'emploi du temps type, puis gérer les exceptions d'une semaine précise : annulation d'un cours, séance exceptionnelle.

— Filtrer les élèves et agir sur une sélection : changer de classe, réinitialiser le mot de passe, exporter en CSV, supprimer.

— Justifier les absences, enregistrer les mesures disciplinaires, supprimer notes, publications et réponses.

2.3 Règles de gestion

— Un élève appartient à une seule classe ; un professeur peut enseigner plusieurs matières dans plusieurs classes.

— Un élève ne consulte que ses propres notes, absences et mesures disciplinaires, et le calendrier de sa classe.

— Un professeur ne consulte que les absences de sa matière dans les classes qu'il enseigne.

— Les profils ne sont modifiables que par l'administrateur.

— Seul l'administrateur justifie une absence.

— Une absence est définie par une date et une plage horaire.

2.4 Besoins non fonctionnels

— Sécurité : mots de passe hachés, accès protégé par jeton, contrôle des rôles.

— Ergonomie : navigation par onglets, thème sombre sobre, calendrier lisible.

— Maintenabilité : architecture en couches, composants React découpés.

— Portabilité : application web, sans installation côté utilisateur.

— Performance : le calendrier est chargé semaine par semaine plutôt qu'en entier.

Sécurité : mots de passe hachés, accès protégé par jeton, contrôle des rôles. Ergonomie : navigation par onglets, thème sombre sobre, calendrier lisible. Maintenabilité : architecture en couches, composants React découpés. Portabilité : application web, sans installation côté utilisateur. Performance : le calendrier est chargé semaine par semaine plutôt qu'en entier.

Chapitre 3

Choix technologiques

Le critère principal a été la maîtrise : avec un délai court, une technologie déjà étudiée vaut mieux qu'une technologie théoriquement meilleure mais nouvelle.

Couche	**Technologie**	**Justification**
Backend	Spring Boot 4.1.1 (Java)	Technologie étudiée en cours, configuration rapide et écosystème complet pour le web, la sécurité et la persistance.
Persistance	Spring Data JPA, Hibernate	Schéma généré depuis les entités et requêtes dérivées du nom des méthodes.
Base de données	MySQL	SGBD relationnel adapté aux nombreuses relations entre entités.
Sécurité	Spring Security, JWT (jjwt 0.12.5), BCrypt	Authentification sans session serveur et mots de passe hachés.
Productivité	Lombok	Réduction du code répétitif : getters, setters et constructeurs.
Frontend	React, Vite	Composants réutilisables et démarrage rapide.
Navigation et HTTP	react-router-dom, axios	Routage côté client et intercepteur ajoutant le jeton aux requêtes.
Outils	Spring Tools Suite 5, Maven, Postman	Environnement de développement, gestion des dépendances et test des endpoints.

Table 3.1 – Technologies utilisées

Alternatives étudiées

— Node.js/Express, Laravel, Django : plus rapides à démarrer pour du CRUD, mais le backend

— Spring Boot était déjà connu, ce qui évitait un apprentissage coûteux.

— H2 à la place de MySQL : plus simple à installer, mais MySQL a été retenu pour se rapprocher d'un contexte réel.

— Create React App : abandonné au profit de Vite, plus rapide et toujours maintenu.

— Héritage JPA à table unique : écarté au profit de la stratégie JOINED (voir section 4.3).

Chapitre 4

Conception

4.1 Architecture générale

L'application suit une architecture en trois niveaux. Une application React s'exécute dans le navigateur et communique en JSON avec une API REST Spring Boot, qui lit et écrit dans une base MySQL.

En développement, Vite relaie les appels /api vers le backend (port 8080), ce qui évite toute configuration CORS. Le backend est organisé en couches : model (entités), repository (accès aux données), service (logique métier), controller (endpoints REST), dto (objets de transfert) et config (sécurité).

4.2 Modèle de données

Le modèle compte 16 entités. Les relations sont toutes des associations plusieurs-à-un portées par des clés étrangères.

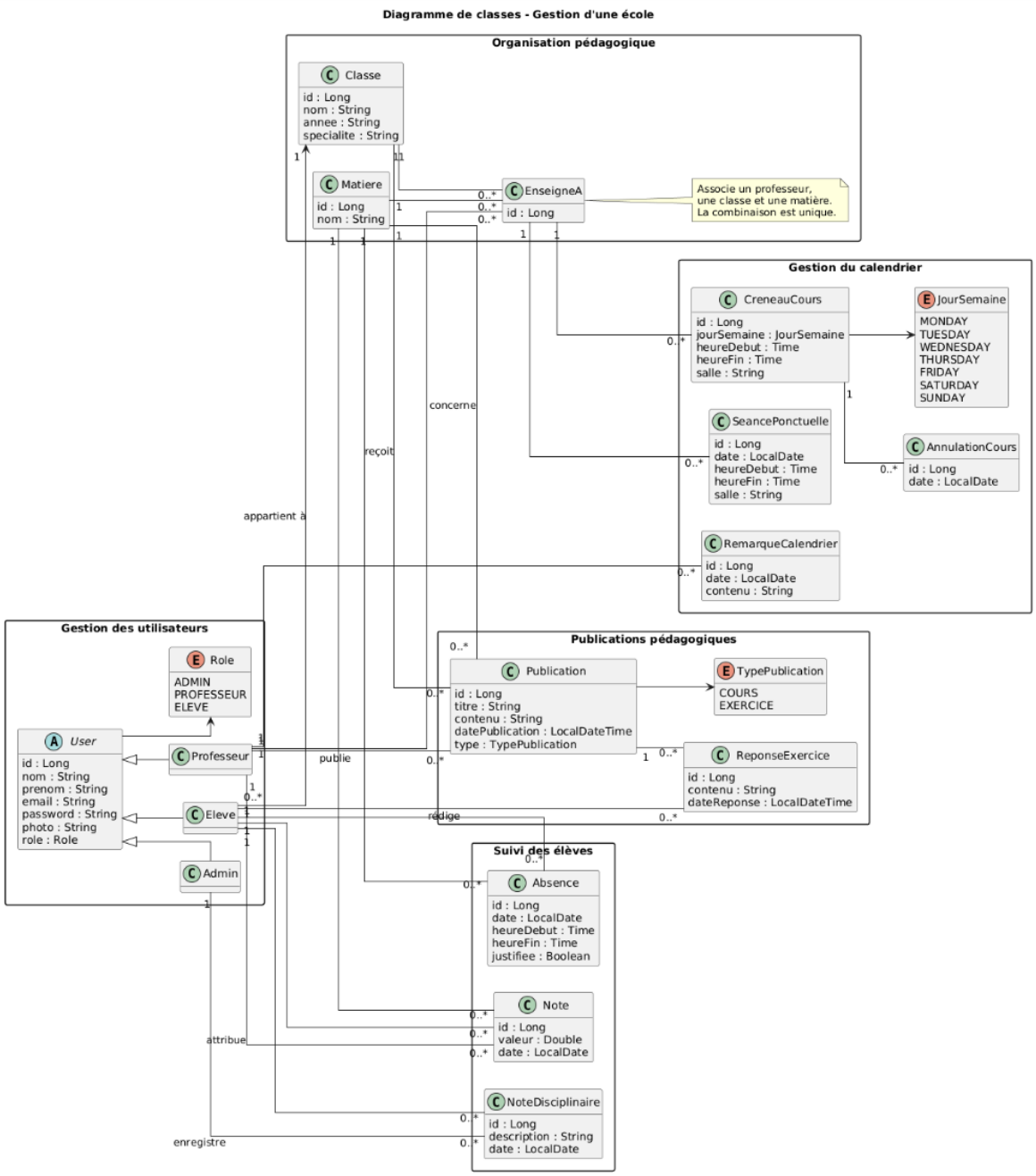


FIGURE 4.1 – Diagramme de classes UML de l'application

TABLE 4.1 – Entités principales du modèle de données

Entité	**Rôle**	**Relations principales**
User	Compte de base : nom, prénom, email, mot de passe haché, rôle et photo	Parent de Eleve, Professeur et Admin.
Eleve	Élève	Appartient à une Classe.

Entité	Rôle	Relations principales
Professeur	Enseignant	Lié à ses classes et matières via EnseigneA.
Admin	Administrateur	Auteur des notes disciplinaires.
Classe	Groupe d'élèves : nom, année et spécialité	Contient des Eleve.
Matiere	Discipline enseignée	Utilisée par EnseigneA, Note, Absence et Publication.
EnseigneA	Affectation d'un professeur à une classe et une matière	Professeur, Classe et Matiere ; combinaison unique.
CreneauCours	Créneau hebdomadaire de l'emploi du temps type	EnseigneA, jour, heures et salle.
AnnulationCours	Annulation d'un créneau à une date précise	CreneauCours et date.
SeancePonctuelle	Séance exceptionnelle à une date précise	EnseigneA, date, heures et salle.
Note	Note d'un élève dans une matière	Eleve, Matiere et Professeur.
Absence	Absence avec date, plage horaire et statut	Eleve et Matiere.
NoteDisciplinaire	Mesure disciplinaire	Eleve et Admin.
Publication	Cours ou exercice publié pour une classe	Professeur, Classe et Matiere.
ReponseExercice	Réponse d'un élève à un exercice	Publication et Eleve.
RemarqueCalendrier	Remarque personnelle d'un professeur	Professeur.

4.3 Héritage JPA

Les trois types d'utilisateurs partagent les mêmes champs de base. La stratégie JOINED a été retenue : une table users contient les champs communs, et chaque sous-classe a sa propre table (eleves, professeurs, admins) reliée par la clé primaire.

Ce choix évite les colonnes vides d'une table unique et garantit l'intégrité par clés étrangères. En contrepartie, chaque création écrit dans deux tables : les comptes doivent donc être créés par l'API, jamais par insertion SQL manuelle.

Professeur

4.4 Conception du calendrier

Le calendrier repose sur deux niveaux. L'emploi du temps type (CreneauCours) décrit la semaine qui se répète. Les exceptions (AnnulationCours, SeancePonctuelle) portent sur une date précise.

Le serveur calcule les cours réels de chaque semaine à partir de ces trois sources et renvoie une liste d'occurrences datées. Cette conception évite de stocker un cours par semaine et par classe, et permet de naviguer librement d'une semaine à l'autre.

4.5 Évolution du modèle

Le modèle initial comprenait des entités pour les clubs et les événements de calendrier. Elles ont été supprimées lors de la réduction du périmètre, et le calendrier a été reconstruit sur le modèle ci-dessus.

Chapitre 5

Réalisation du backend

5.1 Structure

Chaque entité dispose d'un repository qui étend JpaRepository, ce qui fournit les opérations de base sans SQL. Les requêtes courantes sont dérivées du nom de la méthode, par exemple findByEleve ou findByEmail. Les contrôleurs exposent l'API sous le préfixe /api.

5.2 Authentification et autorisation

Le flux d'authentification se déroule en quatre temps :

1. L'utilisateur envoie son email et son mot de passe à `/api/auth/login`.
2. Spring Security compare le mot de passe saisi au hachage BCrypt stocké.
3. Le serveur renvoie un jeton JWT signé (HS256), valable 24 heures, avec l'identifiant, le rôle et le nom.
4. À chaque requête suivante, le filtre JwtAuthFilter lit l'en-tête `Authorization: Bearer`, valide le jeton et charge l'utilisateur dans le contexte de sécurité.

La configuration est sans session (STATELESS) : le serveur ne conserve aucun état entre deux requêtes. Seul /api/auth/** est public. La classe UserPrincipal adapte l'entité User à l'interface UserDetails de Spring Security. Les droits par rôle sont déclarés sur chaque méthode avec @PreAuthorize, par exemple hasRole('ADMIN') pour toute écriture sur les classes.

5.3 Requêtes avec filtres optionnels

La recherche d'élèves accepte quatre filtres facultatifs (nom, classe, année, spécialité). Un filtre absent devient une condition toujours vraie :

@Query("SELECT e FROM Eleve e WHERE " + "(:nom IS NULL OR LOWER(e.nom) LIKE LOWER(CONCAT('%', :nom, '%'))) AND " + "(:classeId IS NULL OR e.classe.id

= :classeId) AND " + "(:annee IS NULL OR e.classe.annee = :annee) AND " + "(:specialite IS NULL OR e.classe.specialite = :specialite)") List<Eleve> rechercher(...) ;

5.4 Calendrier dynamique

Le service CalendrierService reçoit une date, en déduit le lundi de la semaine, puis assemble trois sources : les créneaux de l'emploi du temps type, les annulations de la semaine et les séances ponctuelles. Il renvoie des objets OccurrenceCours contenant la date, les horaires, la salle, un indicateur d'annulation et un indicateur de séance exceptionnelle.

Les endpoints /api/calendrier/classe/{id} et /api/calendrier/prof/{id} vérifient que l'utilisateur consulte bien son propre calendrier.

5.5 Actions en masse

L'administrateur peut déplacer plusieurs élèves vers une autre classe, réinitialiser leur mot de passe ou les supprimer en une seule requête. La suppression s'exécute dans une transaction : elle efface d'abord les réponses, notes, absences et mesures disciplinaires de l'élève, puis l'élève lui-même, sans quoi les clés étrangères bloqueraient l'opération.

5.6 Protection des données dans les réponses JSON

Le champ password de l'entité User est annoté @JsonIgnore : le hachage n'est jamais renvoyé au client, même quand une entité est sérialisée dans une liste.

Chapitre 6

Réalisation du frontend

6.1 Organisation du code

Le frontend est découpé en dossiers selon la responsabilité de chaque fichier :

- pages/ : une page par écran (connexion, tableau de bord de chaque rôle).
- components/ : composants réutilisables (onglets, cartes de statistiques, calendrier), avec des sous-dossiers admin/ et prof/.
- services/ : un fichier par domaine, qui regroupe les appels HTTP.
- context/ : l'état d'authentification partagé. utils/ : fonctions utilitaires, notamment la gestion des dates.

6.2 Authentification côté client

Le contexte AuthContext conserve l'utilisateur connecté et le jeton dans le localStorage, ce qui maintient la session après un rafraîchissement de la page. Un intercepteur axios ajoute automatiquement l'en-tête Authorization : Bearer à chaque requête

Après la connexion, l'utilisateur est redirigé vers le tableau de bord de son rôle. Le composant ProtectedRoute vérifie à la fois la connexion et le rôle : un élève qui saisit l'adresse /admin est renvoyé vers la page de connexion.

6.3 Composants réutilisables

- Tabs : navigation par onglets, utilisée dans les trois tableaux de bord et en sous-onglets.
- StatCard : carte de statistique des pages d'accueil.
- WeeklyCalendar : grille hebdomadaire du lundi au samedi, de 8 h à 19 h. Chaque colonne affiche le jour et la date, chaque cours est positionné selon son horaire réel, et sa couleur dépend de la matière.

— CalendrierNavigable : ajoute les boutons semaine précédente, suivante et « Aujourd'hui », et charge les données de la semaine affichée.

Les cours annulés apparaissent grisés avec la mention « Annulé », et les séances exceptionnelles portent une étiquette dédiée.

6.4 Interface et thème

L'interface utilise un thème sombre en nuances de gris avec du blanc pour le texte, et une couleur d'accent bleue réservée aux actions principales et aux éléments sélectionnés. Les couleurs sont définies une seule fois comme variables CSS, ce qui permet de modifier tout le thème en un point.

6.5 Découpage du tableau de bord professeur

La première version du tableau de bord professeur tenait dans un fichier d'environ 400 lignes, avec plus de vingt états. Il a été découpé en composants (calendrier, publications, notation, détail d'une classe), chacun gérant ses propres données. Le fichier principal est tombé à une soixantaine de lignes, et les formulaires se réinitialisent proprement quand le professeur change de classe.

Chapitre 7

Fonctionnalités par rôle

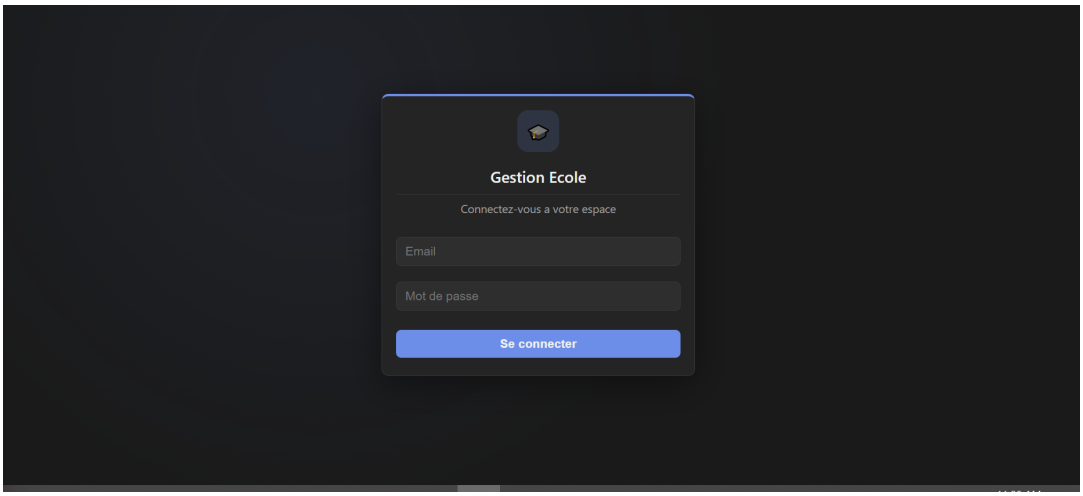


FIGURE 7.1 – Page de connexion

7.1 Administrateur

Le tableau de bord administrateur comporte douze onglets.

Onglet	Fonctionnalités
Accueil	Compteurs : classes, matières et professeurs.
Comptes	Création d'un compte élève, professeur ou administrateur.
Classes	Création et suppression des classes.
Matières	Création et suppression ; affichage des professeurs qui l'enseignent.
Enseignements	Affectation d'un professeur à une classe et une matière.
Créneaux	Définition de l'emploi du temps type.
Calendrier	Navigation par semaine, annulation d'un cours et ajout d'une séance exceptionnelle.
Élèves	Filtres, sélection multiple, changement de classe, réinitialisation du mot de passe, export CSV et suppression.
Profils	Modification du nom, de l'email, du mot de passe, de la photo et de la classe.
Absences	Consultation par élève et justification.
Contenu	Consultation et suppression des notes, publications et réponses.
Discipline	Enregistrement des mesures disciplinaires et historique.

TABLE 7.1 – Fonctionnalités du tableau de bord administrateur

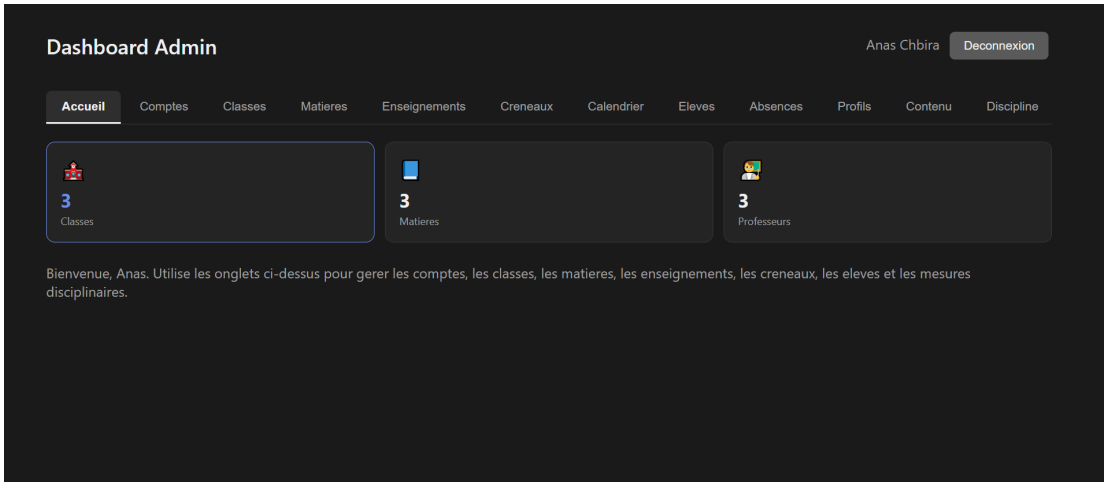


FIGURE 7.2 – Capture d'écran — Tableau de bord administrateur

7.2 Professeur

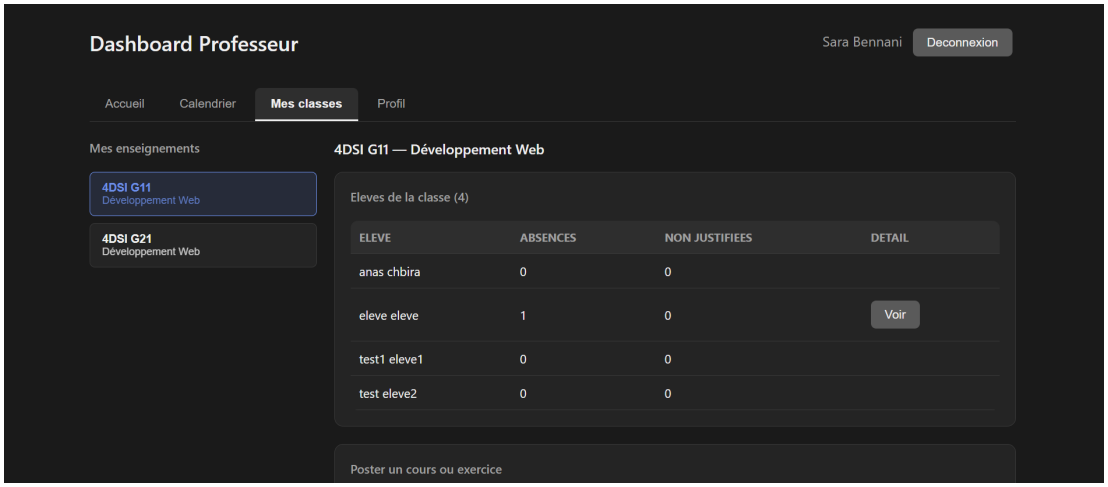


FIGURE 7.3 – Capture d'écran — Page « Mes classes » du professeur

Onglet	Fonctionnalités
Accueil	Nombre de classes, de matières et de créneaux hebdomadaires.
Calendrier	Calendrier navigable et remarques personnelles par date, signalées par une icône.
Mes classes	Liste des enseignements, élèves et absences ; publication de cours et d'exercices ; lecture des réponses ; notation et saisie d'absence.
Profil	Informations du professeur en lecture seule.

TABLE 7.2 – Fonctionnalités du tableau de bord professeur

7.3 Élève

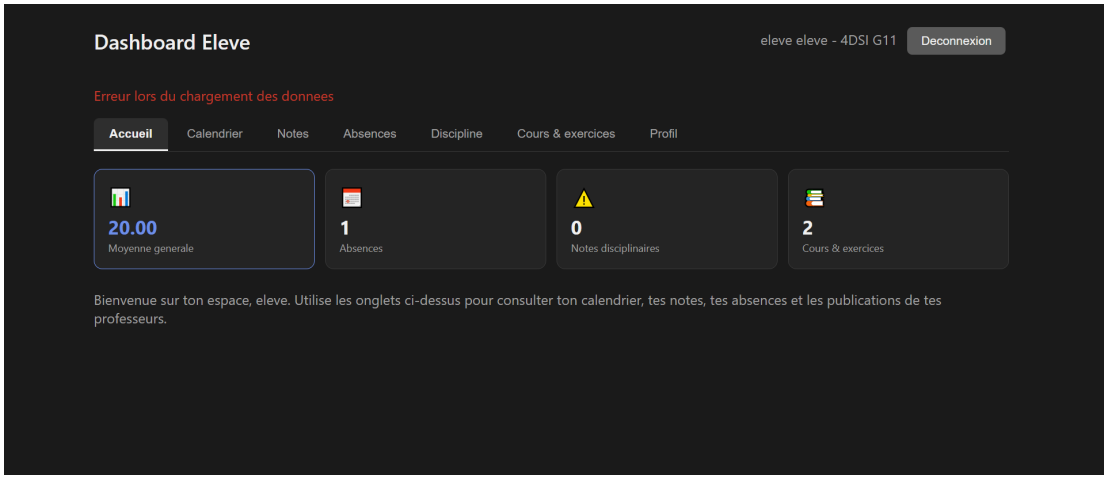


FIGURE 7.4 – Capture d'écran — Tableau de bord élève

Onglet	**Fonctionnalités**
Accueil	Moyenne générale, nombre d'absences, de mesures disciplinaires et de publications.
Calendrier	Calendrier de la classe avec nom du professeur et salle.
Notes	Notes par matière avec la date.
Absences	Absences avec horaires et statut justifié ou non.
Discipline	Consultation des mesures disciplinaires.
Cours et exercices	Publications de la classe et zone de réponse pour chaque exercice.
Profil	Informations en lecture seule ; modification via l'administration.

TABLE 7.3 – Fonctionnalités du tableau de bord élève

Chapitre 8

Sécurité et protection des données

La sécurité repose sur quatre mécanismes complémentaires, puis sur des contrôles de propriété des données.

8.1 Mécanismes en place

Authentification : mot de passe haché avec BCrypt, jamais stocké ni renvoyé en clair.

Jetons JWT : signés, avec une durée de validité de 24 heures ; aucune session n'est conservée côté serveur.

Autorisation par rôle : annotations @PreAuthorize sur les endpoints, par exemple écriture réservée à l'administrateur.

Routes protégées côté client : un utilisateur ne peut pas ouvrir le tableau de bord d'un autre rôle.

8.2 Contrôles de propriété

Le rôle seul ne suffit pas : un élève pourrait changer un identifiant dans une URL pour lire les données d'un autre. Le backend vérifie donc la propriété des données.

— Un élève ne lit que ses propres notes, absences et mesures disciplinaires, et uniquement le calendrier et les publications de sa classe.

— Un professeur ne consulte que les absences de sa matière dans ses classes, et que son propre calendrier.

— Un professeur ne supprime que ses propres publications, notes et remarques.

— La liste des élèves d'une classe et les réponses aux exercices sont réservées aux professeurs et administrateurs.

8.3 Points d'attention connus

Inscription publique : l'endpoint /api/auth/register est ouvert pour permettre de créer le premier administrateur. Il doit être restreint aux administrateurs une fois ce compte créé.

Stockage du jeton : le jeton est conservé dans le localStorage, exposé en cas de faille XSS ; des cookies HttpOnly seraient plus robustes.

Secret de signature : il figure dans le fichier de configuration ; il doit être fourni par une variable d'environnement en production.

Pas de jeton de rafraîchissement : après 24 heures, l'utilisateur doit se reconnecter.

HTTPS : indispensable en production pour protéger les échanges.

Chapitre 9

Tests et validation

Les tests ont été manuels. Chaque endpoint a d'abord été vérifié avec Postman, puis chaque écran a été testé dans le navigateur. Un parcours complet administrateur, professeur puis élève a ensuite été exécuté dans cet ordre, car chaque rôle crée les données utilisées par le suivant.

9.1 Parcours de bout en bout

Table 9.1 – Tests fonctionnels de bout en bout

Étape	Action	Résultat attendu
1	Admin : créer classes, matières, comptes, enseignements et créneaux	Les données apparaissent dans les listes et calendriers.
2	Admin : annuler un cours et ajouter une séance exceptionnelle	Le cours est grisé et marqué « Annulé » pour l'élève et le professeur, uniquement cette semaine.
3	Professeur : publier un cours et un exercice, noter et marquer une absence	Les données sont visibles côté élève et le compteur d'absences augmente.
4	Élève : répondre à l'exercice	Le professeur voit la réponse, classée par élève.
5	Admin : justifier l'absence et ajouter une mesure disciplinaire	Le statut passe à « Oui » côté élève et la mesure apparaît dans l'onglet Discipline.
6	Admin : filtrer, sélectionner et déplacer des élèves, exporter en CSV	Les élèves changent de classe et le fichier conserve les accents.
7	Admin : supprimer un élève qui possède des données	Suppression réussie sans erreur de clé étrangère.

9.2 Tests de sécurité

Test	**Résultat attendu**
Un élève appelle `/api/admin/eleves`	Réponse HTTP 403.
Un élève appelle `/api/classes/1/eleves`	Réponse HTTP 403.
Un élève tente de lire les notes d'un autre élève en changeant l'identifiant	Réponse HTTP 403.
Un élève ouvre l'adresse `/admin`	Redirection vers la connexion.
Vérification du champ `password` dans une réponse JSON	Le champ ne doit jamais être exposé.
Connexion avec un mauvais mot de passe	Message d'erreur clair.

TABLE 9.2 – Tests de sécurité

9.3 Cas limites

Création de deux comptes avec le même email : erreur explicite, sans plantage.

Utilisation d'un jeton expiré : la requête est traitée comme non authentifiée.

Rafraîchissement de la page sur chaque tableau de bord : la session est conservée.

Calendrier : navigation vers des semaines passées et futures, semaine sans cours.

9.4 Limite des tests

Aucun test automatisé n'a été écrit : c'est une piste d'amélioration (section 11).

Chapitre 10

Difficultés rencontrées et solutions

La plupart des blocages venaient de l'environnement ou de détails de configuration, pas de la logique métier. Chacun a été diagnostiqué à partir des logs ou de la console du navigateur.

Table 10.1 – Difficultés techniques et solutions apportées

Problème	**Cause**	**Solution**
Propriétés JWT absentes	Propriétés manquantes dans `application.properties`	Ajout des deux propriétés nécessaires.
Port 8080 déjà utilisé	Ancienne instance encore en cours d'exécution	Arrêt du processus avant de relancer.
Constructeur `DaoAuthenticationProvider()` indéfini	Version récente de Spring Security	Passage direct de `UserDetailsService` au constructeur.
Connexion impossible alors que le compte existe	Ancien jeton expiré envoyé automatiquement	Interception de l'exception : un jeton invalide est ignoré dans le filtre JWT.
Erreur 403 sur un nouvel endpoint	Backend non redémarré	Redémarrage complet.
`profId=undefined` dans les appels	Ancien objet utilisateur dans `localStorage`, sans identifiant	Ajout de l'identifiant à la réponse de connexion, puis déconnexion et reconnexion.
Calendrier élève vide	Cinq appels dans `Promise.all` mais quatre variables de réception	Ajout de la variable manquante.
Hachage du mot de passe visible dans le JSON	Entité sérialisée telle quelle	Annotation `@JsonIgnore` sur le champ.

Problème	**Cause**	**Solution**
Suppression d'élèves refusée	Clés étrangères vers notes, absences et réponses	Suppression transactionnelle des données liées d'abord.
Dates décalées d'un jour	`toISOString()` convertit en UTC	Fonction de formatage local `toIso`.
Calendrier identique chaque semaine	Emploi du temps type sans notion de date	Ajout des annulations et séances ponctuelles, avec calcul par semaine côté serveur.

Enseignements tirés

— Lire le message d'erreur exact (console du navigateur ou logs) avant de supposer la cause.

— Une erreur côté serveur peut apparaître comme un message générique côté interface : ici, une exception de jeton affichait « mot de passe incorrect ».

— Découper tôt un composant qui grossit évite des corrections coûteuses.

Chapitre 11

Limites et perspectives d'évolution

11.1 Limites actuelles

— Pas de pièces jointes sur les cours, exercices et réponses.

— Les photos de profil sont une simple URL : le champ existe, mais aucun téléversement n'est prévu.

— Les chevauchements d'horaires ne sont pas détectés à la création d'un créneau.

— Un compte ne peut pas être supprimé s'il est lié à des données, hors cas des élèves ;

— l'archivage serait préférable à la suppression.

— Les listes ne sont pas paginées.

— Aucun test automatisé.

11.2 Perspectives fonctionnelles

— Téléversement de fichiers (pièces jointes, photos de profil).

— Bulletins de notes au format PDF, moyennes pondérées par matière.

— Notifications pour les annulations, nouvelles notes et absences.

— Import CSV d'élèves pour une inscription groupée.

— Détection des conflits d'horaires et des salles occupées.

— Retour du module de clubs d'élèves prévu à l'origine.

11.3 Perspectives techniques

— Tests unitaires et d'intégration (JUnit, Spring Boot Test) et tests d'interface.

— Jeton de rafraîchissement et cookies HttpOnly.

— Restriction de l'inscription aux administrateurs.

— Pagination côté serveur, et migrations de base de données versionnées (Flyway).

— Déploiement avec HTTPS et configuration par variables d'environnement.

11.4 Accessibilité et inclusion

Une application scolaire s'adresse à des utilisateurs très variés. Plusieurs améliorations rendraient l'interface plus accessible :

— Un mode clair en plus du thème sombre, et le respect des contrastes recommandés par les règles WCAG.

— Une navigation complète au clavier et des étiquettes ARIA pour les lecteurs d'écran, en particulier sur le calendrier.

— Des tailles de texte ajustables et une mise en page adaptée aux téléphones.

— Une interface multilingue, avec l'arabe et l'écriture de droite à gauche en complément du français.

— Des indications qui ne reposent pas sur la couleur seule : le calendrier ajoute déjà les mentions « Annulé » et « Séance exceptionnelle » en plus du style.

Chapitre 12

Conclusion

Ce Projet a abouti à une application web de gestion d'école fonctionnelle, qui couvre les trois profils attendus : administrateur, professeur et élève. Elle centralise les comptes, les emplois du temps, les notes, les absences, les mesures disciplinaires et les supports pédagogiques, avec un accès contrôlé par rôle.

Sur le plan technique, ce projet m'a fait travailler une chaîne complète : modélisation relationnelle avec héritage JPA, API REST sécurisée par JWT, et interface React organisée en composants. La conception du calendrier, qui combine un emploi du temps type et des exceptions datées, a été la partie la plus formatrice.

J'en retiens aussi des méthodes de travail : avancer par étapes vérifiables, lire précisément les messages d'erreur, réduire le périmètre pour livrer un produit abouti, et refactorer quand un composant devient trop gros.

L'application reste perfectible, notamment sur les pièces jointes, les tests automatisés et l'accessibilité. Elle constitue une base saine pour y ajouter ces évolutions.

Chapitre 13

Webographie

Références techniques des outils utilisés (sites officiels) :

- Spring (Boot, Data JPA, Security) : spring.io
- Hibernate ORM : hibernate.org
- JJWT (bibliothèque JWT pour Java) : github.com/jwtk/jjwt
- Project Lombok : projectlombok.org
- MySQL : dev.mysql.com React : react.dev Vite : vitejs.dev
- React Router : reactrouter.com
- axios : axios-http.com
- Postman : postman.com

Chapitre 14

Annexes

14.1 Annexe A : structure du projet

gestion-ecole/ (backend Spring Boot) :

- src/main/java/com/ecole/gestion_ecole/
- model/ entités JPA et énumérations
- repository/ interfaces JpaRepository
- service/ CalendrierService controller/
- contrôleurs REST
- dto/ objets de transfert (requêtes, réponses, OccurrenceCours)
- config/ sécurité : JWT, filtre, UserPrincipal src/main/resources/application.properties

gestion-ecole-frontend/ (React + Vite) :

- src/
- pages/ Login, DashboardAdmin, DashboardProfesseur, DashboardEleve
  - components/ Tabs, StatCard, WeeklyCalendar, CalendrierNavigable

    admin/ composants de l'administrateur
  - prof/ composants du professeur
- services/ appels HTTP par domaine
- context/ AuthContext
- utils/ dates.js

14.2 Annexe B : principaux endpoints de l'API

Table 14.1 – Principaux endpoints de l'API REST

Domaine	Route	Accès
Authentification	`POST /api/auth/login`	
`POST /api/auth/register`	Public.	
Classes	`GET /api/classes`	
`POST, PUT, DELETE`	Connecté ; écriture admin.	
Élèves d'une classe	`GET /api/classes/{id}/eleves`	Professeur, admin.
Matières	`GET /api/matieres`	
`POST, PUT, DELETE`	Connecté ; écriture admin.	
Profils	`GET /api/eleves/me`	
`GET /api/professeurs/me`	Connecté.	
Enseignements	`GET /api/enseigne`	
`GET /api/enseigne/prof/{id}`		
`POST, DELETE`	Connecté ; écriture admin.	
Créneaux	`GET /api/creneaux/classe/{id}`	
`GET /api/creneaux/prof/{id}`		
`POST, DELETE`	Connecté ; écriture admin.	
Calendrier	`GET /api/calendrier/classe/{id}?debut=`	
`GET /api/calendrier/prof/{id}?debut=`	Propriétaire ou admin.	
Exceptions du calendrier	`POST, DELETE /api/calendrier/annulations`	
`POST, DELETE /api/calendrier/seances`	Admin.	
Notes	`GET /api/notes/eleve/{id}`	
`POST, DELETE`	Élève (les siennes), professeur, admin.	
Absences	`GET /api/absences/eleve/{id}`	
`GET /api/absences/classe/{c}/matiere/{m}`		
`POST, PUT /{id}/justifier`	Selon le rôle ; justification admin.	

Domaine	Route	Accès
Mesures disciplinaires	`GET /api/notes-disciplinaires/eleve/{id}`	
`POST, DELETE`	Lecture connecté ; écriture admin.	
Publications	`GET /api/publications/classe/{id}`	
`POST, DELETE`	Élève de la classe ; écriture professeur.	
Réponses	`GET /api/reponses/publication/{id}`	
`POST, DELETE`	Lecture professeur/admin ; envoi élève.	
Remarques	`GET /api/remarques/mes-remarques`	
`POST, DELETE`	Professeur.	
Gestion des élèves	`GET /api/admin/eleves`	
`POST .../changer-classe`		
`POST .../reinitialiser-mot-de-passe`		
`DELETE .../supprimer-en-masse`	Admin.	
Gestion des profils	`GET /api/admin/users`	
`PUT /api/admin/users/{id}`	Admin.	